

بسم الله الرحمن الرحيم



Sudan University of Science and Technology
College of Computer Science and Information Technology

AI-Based Building Damage Assessment in Post-War Sudan
Using Deep Learning and Remote Sensing

A Thesis submitted as a Requirement for the Bachelor's degree

Prepared by:

1. Mohammed Noor Mohammed Ahmed – Information Systems
2. Mohannad Salah Eldin Fadulallah – Software Engineering
3. Omer Abd El-Daim Hassan – Computer Networks
4. Yousif Al-Bushra Mohammed Ahmed – Information Systems

Supervisor: Dr. Mohammed Khidir Mohammed Noor

Submission Date: June 2026

الآية

قال تعالى:

﴿وَلَكِنَّ أَكْثَرَ النَّاسِ لَا يَعْلَمُونَ (٦) يَعْلَمُونَ ظَهْرًا مِّنَ الْحَيَاةِ الدُّنْيَا﴾

صدق الله العظيم

سورة الروم الآيات (6 - 7)

Dedication

This work is dedicated to Sudan.

To the land that taught us the meaning of belonging, and to the people who continue to stand strong despite every hardship, we dedicate this project.

We dedicate it to the brave and resilient Sudanese people, to everyone who faced difficult times with courage and hope, and to those who continued to move forward even when the future seemed uncertain.

We dedicate it to everyone who lost a home filled with memories, and to those who were forced to leave their cities, villages, or country in search of safety. To those who carried Sudan in their hearts wherever they went, even when they could no longer remain on its soil. To the displaced, the migrants, and all those who were separated from their homes and loved ones by war, this project is for you. To those who left behind the streets of their childhood, the warmth of their communities, and the people they cherish, while carrying memories and hope wherever they traveled.

We dedicate this project not only as an academic achievement, but also as a message of loyalty and belonging. It is a humble reminder that the dreams of Sudan are still alive, and that its sons and daughters continue to work, learn, and build for a better future.

This work may not heal the pain caused by war, nor bring back what has been lost, but it carries a sincere promise that a generation of Sudanese people still believes in rebuilding, creating opportunities, and contributing to the recovery of their homeland.

To the souls of those who passed away while dreaming of a safer and better Sudan, and to those who continue to hold on to hope despite all challenges, we dedicate this success and this degree.

And to Sudan — a homeland that lives within us wherever we go, unites us no matter how far apart we may be, and remains the greatest dream in our hearts — we dedicate this work.

المستخلص

تُخلف الحروب والكوارث أضراراً جسيمة في البنية التحتية للمناطق الحضرية، مما يشكل تحدياً أمام المنظمات الحكومية والإنسانية الساعية لإعادة الإعمار؛ حيث يعتمد التقييم التقليدي للأضرار على المسوحات الميدانية البطيئة والمكلفة والمحفوفة بالمخاطر في مناطق النزاع. ورغم تطور تقنيات الاستشعار عن بُعد، إلا أن الحلول البرمجية المتكاملة والمنصات القابلة للتشغيل الميداني المستقل لا تزال محدودة. يقترح هذا البحث ويطور منصة هندسية برمجية متكاملة ومتعددة المراحل لفحص وتقييم أضرار المباني آلياً باستخدام صور الأقمار الصناعية ثنائية الزمن. تم تصميم النظام وفق معمارية منفصلة تتكون من واجهة مستخدم تفاعلية مبنية بإطار العمل (React/Vite) تتيح الرفع المباشر ورسم الخرائط، مدعومة بخلفية برمجية غير متزامنة وفعالة باستخدام (FastAPI)، وقاعدة بيانات مدمجة (SQLite) لتوثيق سجلات القياسات ومصفوفات البيانات التفصيلية بصيغة (JSON) لضمان استمرارية البيانات وتوليد التقارير الميدانية. يمر خط المعالجة البرمجي داخل المنصة بـ "قمع تصفية" متدرج يبدأ بمرحلة الكشف المكاني واستخراج كتل المباني لضمان الشمولية المكانية، يليه مرحلة التصفية الهندسية للوضوء وشبكات الطرق، ثم مرحلة التدقيق الدلالي عبر نموذج رؤية ولغة محلي للتأكد من هوية المنشآت الحضرية، وصولاً إلى مرحلة الاستنتاج المقارن النهائي التي تولد تصنيفات الأضرار مدعومة بتبريرات لغوية تفسيرية باللغة العربية. ويوثق البحث التطور التقني للنظام عبر ثلاث مراحل تجريبية؛ حيث فشلت المرحلة الأولى المعتمدة على نموذج (U-Net) أحادي النموذج في التمييز بين ركام الانقاض وظلال المباني المتغيرة، بينما واجهت المرحلة الثانية المعتمدة على الحوسبة السحابية (Google Colab) قيوداً صارمة في سرعة الشبكة وأخطاء معدل الطلبات (429 HTTP). أدى ذلك للتحويل الهندي نحو المعمارية الحالية التي تعمل محلياً بالكامل (Local Edge Compute) على أجهزة المستهلك العادية باستخدام نموذج رؤية ولغة مكتمل (Qwen2.5-VL:7b) عبر (Ollama) لضمان مجانية التشغيل والعمل دون إنترنت في ظروف الأزمات. تم تقييم أداء المنصة عملياً عبر 50 محاولة تشغيل مؤتمتة على منطقة وسط الخرطوم (السودان) تزامناً مع أحداث أبريل 2023؛ حيث نجح النظام في جرد ومزامنة بيانات 1,611 مبنى، مسجلاً معدل تغطية مكانية بلغ 88.83% ومعدل قبول دلالي قدره 95.70%. ولإثبات الدقة الأكاديمية الصارمة، تم فحص عينة عشوائية مكونة من 163 مبنى ومطابقتها يدوياً مع التقييم البشري الفعلي (Ground Truth)، حيث حققت المنصة دقة تصنيف كلية بلغت 73.62%، مع تسجيل معدل استدعاء قدره 96% في النقاط المباني ذات الأضرار الكبيرة، مما يثبت الكفاءة العملية للمنصة كأداة موثوقة لدعم اتخاذ القرار الإنساني والتخطيط لإعادة الإعمار.

Abstract

Wars and disasters cause extensive urban infrastructure damage, creating severe operational bottlenecks for state and humanitarian organizations prioritizing reconstruction efforts. Traditional damage assessment relies heavily on manual field surveys, a dynamic that remains slow, prohibitively expensive, and hazardous within active conflict zones. While remote sensing technologies and high-resolution satellite imagery have improved spatial monitoring, integrated, production-grade software platforms capable of autonomous field operation remain scarce. To address this gap, this research presents the design, implementation, and empirical evaluation of an integrated, multi-stage software platform for automated building damage assessment using bi-temporal satellite imagery. The system utilizes a decoupled architecture featuring a responsive React/Vite single-page frontend client for dynamic bounding-box rendering, an efficient asynchronous FastAPI backend, and an integrated SQLite database layer to log system telemetry, serialized results metadata arrays within standardized JSON fields, and average response latency profiles. The underlying analytical engine operates through a progressive "Filtering Funnel" that initiates with deep spatial segmentation for comprehensive structural discovery, followed by a physics-based geometric gate to prune road and background noise, and a local vision-language gate for semantic domain verification, culminating in a bi-temporal reasoning core that yields final standardized damage categories alongside natural language text justifications in Arabic. Crucially, this thesis documents a technical evolution across three empirical development phases. The initial single-stage U-Net baseline struggled to distinguish structural rubble from shifting building shadows, while a subsequent cloud-hosted API approach triggered frequent HTTP 429 resource exhaustion errors and network latency bottlenecks. These baseline failure modes necessitated a strategic pivot to the current production framework: a fully autonomous local edge runtime executing a quantized vision-language model (Qwen2.5-VL:7b via Ollama) hosted on consumer-grade hardware to guarantee private, zero-cost inference and offline field survivability during communication blackouts. The platform was rigorously evaluated across 50 production runs mapping the high-density conflict footprint of Central Khartoum, Sudan, centering on the historical events of April 2023. The system successfully processed and committed a cumulative inventory of 1,611 building structures, logging a Spatial Coverage Score of 88.83% and an exceptional Semantic Acceptance Rate of 95.70%. To establish empirical validation, a

micro-level verification dataset of 163 individual structures was cross-verified against manual human ground truth profiles. The evaluation demonstrated an overall weighted platform accuracy of 73.62% and generated a critical True Positive Recall of 96.00% within the major structural damage tier, validating the platform's utility for non-governmental aid allocation and crisis decision-making

Acknowledgment

First and foremost, we thank Allah for giving us the strength, patience, and guidance to complete this work.

We would like to express our sincere appreciation and gratitude to our project supervisor, Dr. Mohamed Khidir Mohamed Noor, who was a constant source of support and guidance throughout this research journey. He generously shared his knowledge and experience with us and never hesitated to give his time, effort, and advice whenever we needed help. His valuable feedback and continuous encouragement played a major role in improving this project and helping us overcome many challenges along the way.

His patience, dedication, and belief in our abilities motivated us to keep moving forward and strive for better results. What he provided went beyond academic supervision; his support left a lasting impact on our learning experience and personal growth. Words alone cannot fully express our gratitude for everything he has done for us, and we will always be thankful for his guidance and support.

We would also like to extend our sincere thanks to all our university professors, whose knowledge, teaching, and guidance helped shape our academic journey and provided us with the foundation needed to complete this project.

Our appreciation also goes to all team members for their cooperation, commitment, and teamwork throughout the development of this project. Their efforts and dedication were essential to its successful completion.

Finally, we would like to thank our families for their patience, understanding, and continuous support. Their encouragement and sacrifices gave us the strength and motivation to complete this work, and we are deeply grateful for everything they have done for us.

Glossary

AI	Artificial Intelligence
CNN	Convolutional Neural Network
VLMs	vision-language models
SAR	Synthetic Aperture Radar
GLCM	Gray Level Co-occurrence Matrix
API	Application Programming Interface
CIM	City Information Modelling
HBIM	Historic Building Information Modelling
BIM	Building Information Modelling
GIS	Geographic Information System
IoT	Internet of Things

Table of Contents

الآية	i
Dedication	ii
المستخلص	iii
Abstract	iv
Acknowledgment	vi
Glossary	vii
Table of Contents	viii
List of Figures	x
List of Tables	xi
Chapter One: Introduction	1
1.1 Background	2
1.2 Problem Statement	2
1.3 Research Motivation	3
1.4 Research Objectives	3
1.5 Scope of the Study	4
1.6 Research Structure	4
Chapter Two: Literature Review	5
2.1 Introduction	6
2.2 Satellite Imagery in Disaster and Conflict Analysis	6
2.3 Deep Learning for Building Damage Assessment	6
2.4 Multimodal AI and Advanced Techniques	7
2.5 Review of Previous Studies	7
Chapter Three: Research Methodology and System Architecture.....	10
3.1 Introduction	11
3.2 Research Methodology.....	11

3.3 System Architecture Design.....	12
3.4 The Multi-Stage Analytical Engine Pipeline	14
3.5 Operational Pipeline Optimization Strategies	16
3.6 Conceptual Filtering Performance and Core Constraints.....	16
3.7 Orchestration Backend Layer and Persistent Storage	18
3.8 Client Visualization Interface	18
3.9 Data Flow and System Orchestration Lifecycle.....	18
Chapter Four: Platform Data, Engineering and Implementation	20
4.1 Introduction	21
4.2 Data Sources, Sampling, and Preprocessing	21
4.3 Production Technical Implementation	24
Chapter Five: Results, Evaluation, and Recommendations	32
5.1 Empirical Verification Methodology & Data Preparation Workflow.....	33
5.2 Mathematical Formulation of Evaluation Metrics	33
5.3 Quantitative System Telemetry & Spatial Metrics	34
5.4 Micro-Level Evaluation: Human-Verified Accuracy Matrix	36
5.5 Automated Damage Assessment Census (Central Khartoum Footprint).....	38
5.6 Qualitative Analysis and Explainability (AI Reasoning Audit).....	40
5.7 Latency Profiling and Edge Inference Metrics	40
5.8 Platform Failure Analysis and Technical Constraints.....	41
5.9 Baseline Exploratory Trials and System Evolution	41
5.10 Future Work and Recommendations.....	42
References and Appendices	44
References	45
Appendices	47

List of Figures

Figure 1: shows the conceptual system architecture.....	13
Figure 2: illustrates this hierarchical multi-stage filtering funnel.....	14
Figure 3: Historical Timeline and Multi-Temporal Satellite Imagery Extraction from Google Earth Pro	23
Figure 4: Relational Data Schema View.....	26
Figure 5: Sample of the structured RESTful API outbound JSON response payload conforming to the custom AnalysisResponse Pydantic contract.....	27
Figure 6: Frontend dashboard image upload interface showing pre-conflict and post-conflict satellite imagery fields with validation-guarded project parameters.....	28
Figure 7: Interactive analytics dashboard rendering the multi-scale damage map overlay with percentage-positioned color-coded boxes alongside damage charts and quality progress metrics.	29
Figure 8: presents a horizontal bar chart mapping these autonomous global platform performance indices.....	36
Figure 9: shows the alignment via a color-coded confusion matrix heatmap.	38
Figure 10: highlights these distribution metrics using a structured damage census chart.	39

List of Tables

Table 1: Conceptual Data Reduction and Candidate Refinement Workflow	17
Table 2: outlines technical components, versions, and specialized architectural roles ..	24
Table 3: Platform RESTful API Specifications	27
Table 4: Local Infrastructure Deployment Profiles	30
Table 5: Experimental Hardware Specifications	31
Table 6: Global System Performance and Spatial Quality Metrics	35
Table 7: Micro-Level Damage Classification Performance Matrix	37
Table 8: Automated Damage Assessment Census (Central Khartoum)	39

Chapter One: Introduction

1.1 Background

In recent years, the significant increase in armed conflicts and natural disasters globally has created an urgent need for practical and scalable damage assessment methods. Traditional field surveys are often expensive, time-consuming, and difficult to execute when local security conditions deteriorate. To address these limitations, remote sensing technologies and high-resolution satellite imagery provide an effective alternative for monitoring affected regions, allowing researchers and decision-makers to identify structural damage without requiring immediate on-site deployment.

At the same time, computer vision frameworks—particularly deep learning architectures—have become widely adopted for automated image analysis. While existing frameworks such as xView2 demonstrate the capability of deep networks in disaster response, relying on a single model often fails to maintain consistent classification accuracy across different urban environments. To address this limitation, this research develops a multi-stage, pipeline-based framework rather than relying on a single neural network, aiming to improve both structural detection recall and overall system reliability. Using central Khartoum, Sudan, as a primary case study, this project develops an integrated full-stack software system that processes spatial damage metrics and visualizes the compiled data through an interactive visualization dashboard.

1.2 Problem Statement

Accurate structural damage assessment in active conflict zones presents severe operational and technical challenges. Conventional manual inspections are highly localized, hazardous, and logistically slow, making them impractical for rapid emergency intervention and resource allocation. Modern automated remote sensing approaches leverage established models like xView2; however, their reliance on a single network architecture limits their effectiveness across complex, high-density urban landscapes. This drop in performance occurs because building geometries, architectural materials, and surrounding environmental factors vary significantly across different municipal sectors. Consequently, there is a clear need for an adaptable, multi-tiered framework capable of classifying structural destruction incrementally. This research addresses this gap by developing a multi-stage, AI-based software application that enhances damage assessment quality through progressive data filtering and multi-model integration.

1.3 Research Motivation

The limitations identified in contemporary automated assessment tools serve as the primary driver for this research. Although single-model frameworks generate acceptable performance metrics under controlled settings, they often lack the generalization capabilities required for complex, real-world scenarios. Within dense urban areas—particularly post-conflict zones—variations in building density, roofing configurations, and satellite imagery angles introduce significant visual noise. This vulnerability requires a comprehensive system that integrates multiple analytical processing layers instead of consolidating the entire computational workload onto a single model. This research aims to develop a multi-stage artificial intelligence system that improves damage classification precision through structured data refinement. By connecting these backend computer vision pipelines with a graphical user interface, the platform clarifies how spatial analytics are delivered, helping humanitarian agencies and reconstruction planners access actionable findings.

1.4 Research Objectives

This research focuses on designing, implementing, and evaluating a reliable, multi-staged system for automated building damage assessment using bi-temporal satellite imagery and deep computer vision models. To achieve this goal, the study focuses on the following technical objectives:

1. Designing and deploying a multi-stage artificial intelligence pipeline to detect, segment, and analyze structural damage from high-resolution satellite imagery.
2. Improving multi-class damage classification accuracy by leveraging an ensemble-based pipeline of specialized analytical stages rather than a single neural network.
3. Implementing an asynchronous backend architecture to manage data ingestion, orchestrate processing workflows, and ensure stable communication across distinct system layers.
4. Developing an interactive user-facing frontend interface that allows operators to navigate spatial layers and inspect analytical results clearly.
5. Evaluating end-to-end platform performance against human-verified ground truth profiles to establish a clear baseline for system accuracy and future improvements.

1.5 Scope of the Study

This thesis focuses on the automated assessment of structural building damage within affected urban environments using bi-temporal satellite imagery and deep learning methodologies. The geographical scope is restricted to the central urban layout of Khartoum, Sudan, selected due to its structured planning which provides a suitable baseline for aerial image analysis during the April 2023 timeline. Temporally, the dataset evaluates high-resolution spatial scenes captured immediately before and after the onset of the conflict. The software architecture is implemented and tested as a functional pilot prototype rather than a fully operational production system, prioritizing the verification of the multi-stage pipeline's core capabilities. Secondary engineering components, such as long-term financial cost estimation for reconstruction or applying the system to rural topographies, fall outside the scope of this study.

1.6 Research Structure

The structure of this thesis document is organized into four subsequent chapters:

- Chapter 2: Literature Review establishes the theoretical foundations of the domain, surveying prior work in remote sensing, deep convolutional networks, and multimodal AI architectures while isolating the specific technical gaps our pipeline aims to resolve.
- Chapter 3: Research Methodology and System Design outlines the overall architectural framework of our solution, detailing the mechanics of the multi-stage AI pipeline, the backend processing engine, and the frontend client interface alongside the parameters governing spatial discovery and semantic verification.
- Chapter 4: Data Preparation and System Implementation details the engineering execution of the platform, explaining satellite data acquisition, image tile preprocessing, API gateway design, database synchronization, and client-side visualization components.
- Chapter 5: Results, Evaluation, and Conclusion presents a quantitative and qualitative analysis of platform performance, cross-validating system metrics against manual ground truth matrices, analyzing technical limitations, and proposing recommendations for future development.

Chapter Two: Literature Review

2.1 Introduction

This chapter examines the existing body of research on automated building damage assessment using artificial intelligence and aerial imagery, establishing the theoretical and technical foundations for the proposed system. The discussion opens by analyzing the role of satellite imagery in documenting destruction caused by natural disasters and armed conflicts. It then addresses the deep learning architectures commonly applied to image analysis and structural damage detection tasks. The chapter also traces recent advances in multimodal AI and its growing utility in complex visual reasoning scenarios. Finally, the chapter surveys prior work on building damage assessment across a range of methodologies before identifying the specific gaps and limitations that the proposed framework targets.

2.2 Satellite Imagery in Disaster and Conflict Analysis

Satellite imagery has become a primary data source for monitoring areas devastated by disasters or armed conflicts. This prominence is largely because remote sensing delivers wide-area coverage without requiring physical access to the affected zone—access that is frequently impossible due to active security threats or severe structural destruction.

High-resolution imagery captures fine structural details of buildings, bridges, and urban infrastructure. Comparing pre- and post-event image pairs exposes damage patterns that would otherwise demand weeks of ground-level survey work, making this approach highly valuable in time-critical emergency response scenarios. However, manual analysis of these images is slow, labor-intensive, and prone to inconsistency. Variations in image resolution, atmospheric conditions, and lighting angles introduce ambiguity that human analysts struggle to resolve systematically. These limitations have pushed the field toward automated, learning-based analysis pipelines.

2.3 Deep Learning for Building Damage Assessment

Deep learning has established itself as a standard paradigm for satellite image analysis and structural damage detection. This shift has occurred primarily because deep networks learn complex spatial patterns directly from raw pixel data, often outperforming traditional feature-engineered methods.

Convolutional Neural Networks (CNNs) form the core of most architectures in this domain. Their hierarchical feature extraction makes them well-suited for recognizing building footprints and classifying damage severity across large image tiles. Semantic segmentation

models, such as U-Net, extend this capability by producing pixel-level building masks that enable the precise delineation of affected structures.

The dominant analytical strategy pairs pre-event and post-event imagery to run change detection and isolate damaged regions. The xView2 benchmark operationalized this approach at scale, defining a four-tier damage scale ranging from no damage to complete destruction. However, most systems built on this benchmark rely on a single trained model, a design choice that introduces a risk of fragility. Shifts in image quality, shadow patterns, local architectural styles, or regional building materials can significantly degrade model accuracy when deployed outside the specific training distribution. This vulnerability motivates the development of ensemble-based or multi-model systems, where combining several specialized models can improve consistency and generalization across diverse operating environments.

2.4 Multimodal AI and Advanced Techniques

Progress in AI research has produced a class of models capable of reasoning jointly over images and natural language. These vision-language models (VLMs) do not simply classify or segment; they interpret visual content and generate contextual descriptions that enrich downstream analysis.

Models such as LLaVA and Gemini have demonstrated strong performance on tasks demanding holistic scene understanding rather than narrow pattern matching. In the context of damage assessment, these models can be leveraged to validate whether detected regions correspond to actual built structures, and to characterize damage with a level of semantic detail that pixel-level classifiers cannot produce alone.

However, integrating these components—detection networks, segmentation models, and vision-language reasoning—into a single, coherent system remains technically non-trivial. The resulting system must be accurate, generalizable, and practical enough for real-world deployment under disaster-response constraints. This intersection of requirements defines the technical challenge that this work addresses.

2.5 Review of Previous Studies

Recent years have seen an increase in research applying artificial intelligence, computer vision, and remote sensing to automate building damage assessment following natural disasters and armed conflicts. These studies have drawn on a wide range of techniques—including CNNs, Siamese networks, semantic segmentation, super-resolution enhancement,

vision-language models, and multi-source data fusion combining optical and Synthetic Aperture Radar (SAR) imagery. However, few systems consolidate these capabilities into a unified, deployable pipeline (Fujita et al., 2017; Ghandour & Jezzini, 2021; May et al., 2022; Li et al., 2023; Azimi et al., 2024).

2.5.1 Early Technological Foundations

Fujita et al. (2017) built one of the earliest CNN-based frameworks for detecting washed-away buildings following the 2011 Great East Japan Earthquake and tsunami. Their system processed pre- and post-event aerial image pairs and achieved 94–96% classification accuracy on the ABCD dataset, which they assembled specifically for that study. Ghandour and Jezzini (2021) took a different approach, developing an automated pipeline grounded in shadow deformation analysis and Gray Level Co-occurrence Matrix (GLCM) texture features extracted from high-resolution GeoEye-1 satellite imagery over Zabadani, Syria. That system reached 81.25% accuracy on binary damaged/undamaged classification, a competitive result given the visual complexity of conflict-zone imagery.

2.5.2 Deep Learning at Scale

More recent work has pushed both classification accuracy and architectural sophistication further. May et al. (2022) benchmarked several architectures—including UNet, EfficientNet, and SE-ResNet—against the xView2 and Haiti datasets, achieving building segmentation F1-scores up to 0.834 alongside a four-class damage classification spanning no damage, minor, major, and destroyed categories; destroyed buildings yielded an F1-score approaching 0.70. The system proposed in this thesis adopts the same pre- and post-event assessment paradigm established by xView2, extending it into a practical application featuring model fine-tuning, API-based deployment, and structured result visualization.

Azimi et al. (2024) demonstrated that fusing CNN and UNet models with both SAR and optical inputs accelerates damage mapping, validating their method on imagery from the 2023 Türkiye–Syria earthquake. Al-Mutairi et al. (2024) reported 86% accuracy in binary classification and 79% in three-tier severity grading (minor, moderate, and severe) using segmentation models on sub-meter conflict imagery. Furthermore, Al-Qahtani et al. (2024) reinforced a complementary point: time-series imagery paired with contextual scene features improves the detection of high-severity destruction events that single-image models tend to miss.

2.5.3 Reconstruction-Oriented and Multimodal Approaches

Some recent literature has pushed beyond automated detection alone. Chashyn et al. (2023) designed the City Intelligent Modelling (CIM) framework, combining Historic Building Information Modelling (HBIM), BIM, GIS, and IoT under an AI layer to support digital reconstruction workflows for war-damaged heritage buildings. This approach serves as a reminder that damage assessment fields into longer-term recovery pipelines. Other studies have experimented with video super-resolution coupled with vision-language models on drone footage, showing that richer input representations can raise damage classification performance on visually degraded imagery (Chashyn et al., 2023).

2.5.4 Identified Research Gaps

The existing body of work described above is technically valuable but often narrow in operational scope. Most studies concentrate almost exclusively on raw detection and classification accuracy, treating result delivery and system usability as secondary concerns. Very few provide end-to-end pipelines that package deep learning outputs—such as damage classes, segmentation masks, and visual overlays—into accessible interfaces for non-technical operators.

This gap becomes acute in post-conflict, resource-constrained environments like Sudan, where decision-makers require interpretable, actionable outputs rather than raw model predictions. The framework developed in this thesis addresses this limitation: it is designed not only to assess structural damage accurately, but to expose those results through a lightweight API that non-specialist users can navigate without prior machine learning training.

Chapter Three: Research Methodology and System Architecture

3.1 Introduction

In the immediate aftermath of natural disasters or armed conflicts, rapid situational awareness is critical for effective humanitarian response and resource allocation. High-resolution satellite imagery provides an expansive aerial view that theoretically enables the assessment of structural damage across entire urban sectors within minutes. However, the massive volume of high-resolution geospatial data introduces a significant processing bottleneck, as manual visual inspection by human analysts cannot scale to meet urgent operational timelines.

Traditional automated computer vision approaches to damage detection face two primary technical limitations:

- **The structural complexity of debris fields:** This presents a significant challenge; standard pixel-level classification models frequently confuse scattered rubble piles, shifting environmental shadows, and uprooted vegetation with actual building footprints.
- **A critical context gap in conventional frameworks:** While detecting radiometric or spectral discrepancies between pre- and post-event co-registered imagery is computationally straightforward, understanding the semantic meaning of those transformations remains difficult. Distinguishing a structurally collapsed roof from regular urban variations requires high-level cognitive inference that flat machine learning architectures lack.

To address these limitations, this chapter details a comprehensive research methodology and an integrated system architecture engineered for high-precision structural damage assessment. The proposed methodology shifts from a standalone model-centric approach toward a decoupled, systems-oriented framework. This pipeline integrates a multi-stage artificial intelligence filtering funnel with dedicated backend orchestration and interactive frontend layers into a unified operational platform.

3.2 Research Methodology

This research utilizes a systematic, empirical methodology designed to optimize predictive performance across dense and complex urban environments. Preliminary testing demonstrated that relying on an isolated deep learning model for end-to-end damage classification yields high false-positive rates due to variations in aerial imagery quality and structural shadows. Consequently, the methodology employs a tiered, multi-stage software

architecture that refines structural predictions through incremental data reduction and cascading contextual verification.

The operational research methodology is structured into three consecutive phases:

1. **Inbound Ingestion and Spatial Preprocessing:** Collecting bi-temporal satellite image pairs, validating data integrity, and executing pixel normalization to ensure spatial and resolution consistency across datasets.
2. **Multi-Stage Pipeline Analysis:** Routing localized image coordinates through a structured analytical funnel that sequentially isolates building boundaries, filters out geometric terrain noise, verifies urban semantic domains, and reasons over temporal structural damage states.
3. **Geospatial Visualization and Reporting:** Generating verified assessment logs and interactive vector coordinate overlays, which are subsequently delivered to an analytical user dashboard for decision-makers.

By segmenting the workflow into independent yet interconnected operational layers, the methodology balances automated multi-class classification accuracy with practical field deployment utility for post-conflict crisis response.

3.3 System Architecture Design

The platform is engineered using a decoupled architectural pattern. Separating the core system components into three independent layers ensures modular scalability, system flexibility, and ease of future framework expansions.

1. **The Analytical Intelligence Engine:** A tiered processing pipeline responsible for automated object localization, rule-based geometric filtering, and multimodal bi-temporal change reasoning.
2. **The Orchestration Backend Layer:** The central gateway and control hub that manages internal state transitions, executes image transformations, coordinates API routing, and guarantees data persistence through a structured relational database.
3. **The Client Visualization Interface:** An interactive, web-based single-page application dashboard allowing operators to upload raw image pairs, track pipeline execution telemetry, and review granular damage distribution census indices.

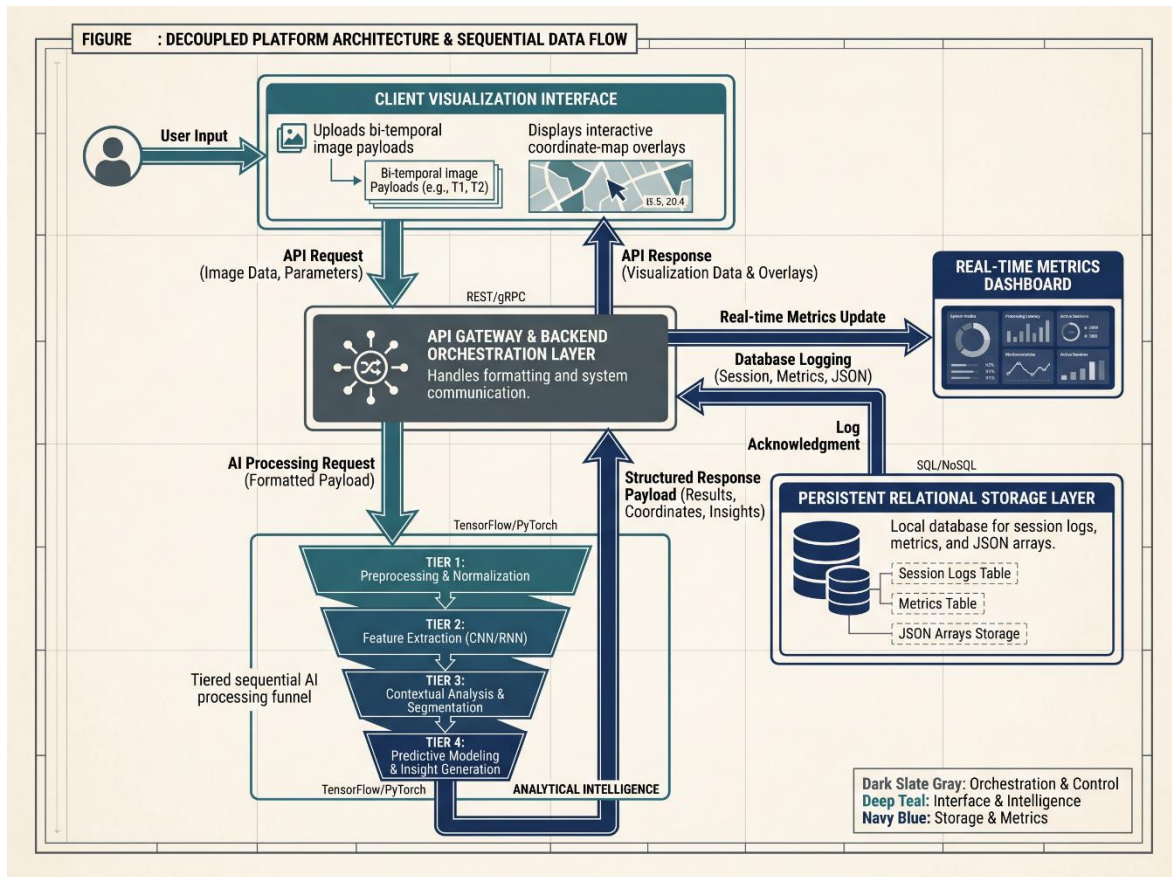


Figure 1: shows the conceptual system architecture

The decoupled data flow across these three primary layers ensures that heavy inference tasks within the analytical engine do not introduce interface latency or block synchronous user transactions on the frontend client.

3.4 The Multi-Stage Analytical Engine Pipeline

The analytical pipeline represents the core technical contribution of this architecture. It is engineered as a progressive data filtering funnel, where semantic depth and contextual intelligence increase sequentially as the volume of candidate building objects decreases. This cascading design mimics the deductive reasoning workflow employed by professional human imagery analysts.

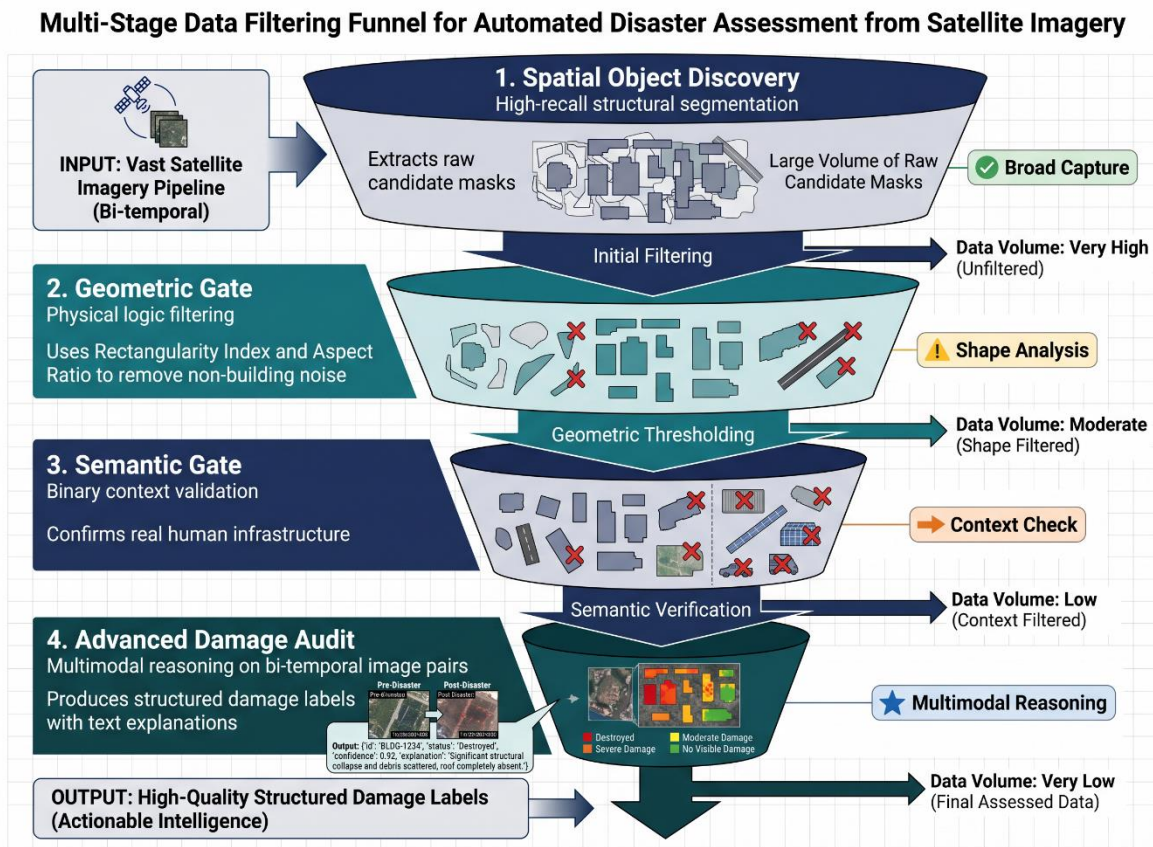


Figure 2: illustrates this hierarchical multi-stage filtering funnel

The progressive contraction of architectural candidates and semantic data reduction from initial object discovery to advanced damage audit reasoning.

Stage 1: Spatial Object Discovery and Geometric Filtering

The pipeline prioritizes high structural recall in its initial phase to minimize the risk of omitting valid building structures. The pipeline initializes a deep spatial segmentation layer that extracts raw instance masks across complex urban topologies, establishing an all-inclusive pool of structural candidates.

To optimize down-pipeline computational performance, a custom geometric shape filter is directly integrated into this discovery stage. This layer automatically evaluates the physical shape properties of the extracted masks to eliminate non-building entities. Because human infrastructure typically exhibits predictable physical and geometric dimensions, the pipeline filters out natural background elements and roads using two structural indices:

- **The Rectangularity Index:** Evaluates the ratio of an object's segmented pixel area to the total area of its minimum bounding envelope rectangle. Solid architectural blocks maintain a high rectangularity profile, whereas scattered rubble fields, vegetation patches, and dynamic shadows display highly irregular geometric shapes.
- **The Aspect Ratio Index:** Measures the proportionality between the length and width of the extracted masks. This index identifies and eliminates highly elongated geometric features, such as linear road networks, retaining walls, and highway boundaries, which general segmentation models frequently misidentify as contiguous structural footprints.

Stage 2: Semantic Domain Verification

The geometrically vetted building candidates are subsequently routed to a localized context-domain validation layer. This semantic verification gate processes each cropped structural bounding box individually to resolve a binary classification question: "Does this visual candidate represent a genuine man-made building structure?"

By executing this localized assessment, the stage successfully prunes remaining natural terrain variations, agricultural features, and false-positive shapes before the bi-temporal change comparison occurs, acting as an automated quality control layer.

Stage 3: Bi-Temporal Structural Audit and Reasoning

The final stage of the analytical engine executes advanced multi-class change detection and visual reasoning across the conflict timeline. The pipeline simultaneously injects co-registered pre-conflict and post-conflict structural image patches into a multimodal reasoning engine.

The engine evaluates structural transformations, surface textures, and boundary degradation over the temporal gap. The output of this stage is a structured data schema containing a standardized multi-class damage classification label—categorized as `no_damage`, `major_damage`, or `destroyed`—coupled with a natural language text justification written in Arabic that explains the specific physical structural modifications observed by the engine.

3.5 Operational Pipeline Optimization Strategies

To ensure that the multi-stage analytical engine can scale to process large-scale urban imagery inputs without causing server timeouts or exceeding hardware memory limits, the platform implements dedicated backend optimization and batch processing strategies. The execution framework transitions from isolated single-object operations toward combined batch execution matrices, utilizing the following core optimization mechanisms:

- **Automated Batch Compounding:** The pipeline optimizes computational throughput by employing an image compounding strategy within its execution core. Instead of dispatching independent, sequential inference requests for every individual building candidate—which would introduce massive API network overhead and GPU initialization latency—the backend automatically stitches multiple bi-temporal building crops onto a single, compiled multi-scale canvas layout grid.
- **Dynamic Matrix Scaling:** Because extracted building segments vary drastically in their layout dimensions, a custom normalization algorithm dynamically scales the matrix boundaries of all processed crops while strictly preserving their native aspect ratios. A single, comprehensive instruction prompt guides the inference engine to evaluate the compiled canvas array and return a unified, nested JSON data structure. This execution strategy significantly reduces total pipeline processing latency and eliminates redundant network handshakes.

3.6 Conceptual Filtering Performance and Core Constraints

The multi-stage design requires a careful architectural balance between high spatial recall at the discovery layer and strict semantic precision at the reasoning layer. To evaluate the theoretical boundaries of the system prior to field deployment, a structured data reduction framework and a systematic analysis of core technical constraints were established.

3.6.1 Quantitative Refinement Framework

Table 1 outlines the cascading data reduction and structural refinement workflow across the independent levels of the multi-stage pipeline:

Pipeline Processing Stage	Target Dataset State	Primary Architecture Role
Stage 1: Spatial Discovery & Geometric Filtering	Raw Extracted Candidate Pool	Structural Mask Tracking, Boundary Localization, Shape Index Pruning (High Recall)
Stage 2: Semantic Gate	Semantically Vetted Subset	Visual Context Domain Verification, Non-Building Noise Elimination, Quality Control
Stage 3: Advanced Audit	Final Damage Inventories	Automated Conflict Auditing, Multi-Class Damage Labeling, Linguistic Justification

Table 1: Conceptual Data Reduction and Candidate Refinement Workflow

3.6.2 Qualitative Reasoning Transparency

A primary operational benefit of this multi-tiered architecture is the generation of explainable outputs. Traditional deep learning classifiers function as "black-box" systems, outputting an abstract numerical class probability without explicit contextual justification. In contrast, this platform produces human-interpretable linguistic descriptions. This structural transparency is critical for verification workflows, allowing humanitarian aid organizations to audit automated assessments before deploying field resources.

3.6.3 Systemic Conceptual Constraints

Despite the modular architecture, specific domain limitations and technical constraints are recognized within the system design:

- **Visual validation ambiguity:** This remains a factor, as general-purpose vision-language models can occasionally exhibit semantic validation biases, verifying highly ambiguous geometric shapes as valid human infrastructure.
- **Temporal and spectral shifts:** These present an inherent challenge; substantial variations in satellite sun angles, atmospheric haze, seasonal shadows, or sensor calibrations between pre- and post-conflict imagery passes introduce localized radiometric noise that can challenge the stability of general visual reasoning engines.

3.7 Orchestration Backend Layer and Persistent Storage

The orchestration backend layer serves as the central processing unit and state manager of the platform. It coordinates data movement across the system boundaries, validates inbound image payloads, and executes image transformations. Upon receiving an image payload from the client, the backend executes matrix normalization, coordinate co-registration, and format adjustments.

To transition the analytical pipeline into a persistent operational platform, an embedded relational database engine is integrated directly into the backend core. Every automated analysis run is captured within a comprehensive database schema. This schema permanently logs vital execution and spatial telemetry, including spatial coverage scores, processing latency profiles, semantic acceptance metrics, and absolute damage distribution ratios. Granular building bounding-box coordinates and generated textual reasoning summaries are serialized as compressed data arrays within the database schema. This approach guarantees absolute traceability, minimizes disk file I/O latency, and provides a secure data foundation for automated report generation.

3.8 Client Visualization Interface

The frontend client interface bridges the gap between complex multi-stage AI pipelines and practical, field-level decision-making. Operators can upload raw pre- and post-conflict satellite imagery through a web-based portal. Once backend processing completes, the frontend maps the responsive data payload into interactive visualization dashboards. The client renders precise vector segmentation boundaries via color-coded thematic layers—displaying green indicators for intact structures, orange for major damage, and red for destroyed elements—while displaying the engine's Arabic textual justifications side-by-side with the geospatial imagery.

3.9 Data Flow and System Orchestration Lifecycle

A data payload follows a strict, circular operational lifecycle through the system boundaries, ensuring data consistency and transactional integrity across all layers. The end-to-end data lifecycle is executed through five consecutive steps:

1. Inbound Submission and Upload: The operator uploads co-registered pre- and post-event satellite image pairs via the single-page client dashboard interface.
2. Ingest Validation and Preprocessing: The frontend transmits the payload to the backend API gateway, where the system executes file format verification, resolution checking, and pixel matrix normalization.
3. Pipeline Orchestration and Relational Logging: The backend initializes a new analysis session record within the relational database and feeds the normalized imagery matrices into the multi-stage AI funnel, managing data transitions across all filtering layers.
4. Outbound Structuring and Serialization: The analytical intelligence engine compiles the multi-class classification arrays and Arabic textual reasoning justifications into a structured schema, subsequently updating the corresponding relational database rows.
5. Geospatial Visualization: The backend sends the completed data response back to the client interface. The frontend application parses the payload to render interactive visual vector overlays and aggregated metric charts.

Chapter Four: Platform Data, Engineering and Implementation

4.1 Introduction

This chapter details the technical implementation and operational deployment of the building damage assessment platform. The theoretical architecture and multi-stage pipelines established in Chapter Three are translated into a functional software system. This integration bridges computer vision concepts with data infrastructure and asynchronous web engineering.

The implementation details are structured into two comprehensive core domains: the geospatial data infrastructure, which focuses on satellite sourcing and preprocessing; and the production software architecture, which manages the application and machine learning orchestration layers.

4.2 Data Sources, Sampling, and Preprocessing

4.2.1 Satellite Imagery Collection and Spatial Extraction

High-resolution satellite imagery was collected utilizing Google Earth Pro's historical imagery timeline feature. The geospatial data gathering targeted the central urban grid of Khartoum, Sudan, centering specifically around the conflict timeline initiated on April 15, 2023. This historical date serves as a definitive temporal baseline for bi-temporal image pair matching, capturing clear pre-conflict configurations alongside subsequent post-conflict structural transformations.

The spatial extraction workflow targeted geographic coordinates representing diverse urban textures and structural typologies. The sampled areas include high-density residential blocks, commercial markets, administrative complexes, and public infrastructure. For each targeted urban sector, strict bi-temporal alignment was enforced; every pre-event imagery frame exactly mirrors the geographic bounding box window and spatial coordinates of its corresponding post-event frame, preventing spatial skewing during structural analysis.

4.2.2 Study Area Description

The central urban grid of Khartoum serves as the primary evaluation site for the platform. This location was selected due to three primary criteria: its high administrative density, its complex architectural layouts, and the availability of clear satellite documentation regarding structural transformations during the conflict period.

The evaluation sector encompasses dense mixed-use building footprints, varied roofing materials (including concrete and corrugated metal), and irregular non-Western architectural

patterns. This topological variety introduces substantial radiometric noise, spectral reflections, and shifting shadow variations into the image dataset. Consequently, this environmental complexity serves as a rigorous baseline to stress-test the platform's multi-stage filtering and reasoning pipeline, validating system performance under unconstrained real-world deployment conditions.

4.2.3 Automated Preprocessing Workflow

Raw remote sensing imagery frequently exhibits atmospheric haze, sensor calibration drifts, and minor geometric offsets that can degrade downstream inference layers. To ensure clean, normalized inputs for the cascading analytical tiers, a structured, code-driven preprocessing workflow was implemented:

1. **Image Tiling Matrix Separation:** Large-scale satellite scenes are programmatically divided into smaller, computationally manageable patch matrices, optimizing local graphics hardware memory layouts during segmentation.
2. **Pixel Matrix Normalization:** Contrast and pixel intensity distributions are standardized across both the pre- and post-event image matrices, neutralizing radical lighting differences caused by variations in satellite acquisition times.
3. **Resolution Standardization:** Input image frames are scaled to match the fixed tensor dimensions required by the spatial discovery layers, using bilinear interpolation to preserve native aspect ratios.
4. **Bi-Temporal Spatial Registration:** OpenCV-based affine transformations are executed to co-register and align the coordinate grids of pre- and post-conflict images, ensuring that any detected spatial change reflects genuine structural transformation rather than coordinate misalignment.

4.2.4 Data Sampling and Quality Control Methodology

A controlled sampling strategy was applied across the Central Khartoum study zone to establish a verifiable dataset for platform verification. A structured selection process identified discrete structural targets visible in both temporal windows. The research compiled an evaluation dataset consisting of approximately 100 to 150 independent building blocks, representing a statistically viable yet computationally manageable sample envelope for localized system testing. To maintain absolute geographical consistency and minimize false-positive detections, strict quality control constraints were enforced during snapshot export:

- **Sensor View Normalization:** Tilted satellite viewing perspectives can cause extreme geometric distortion and artificially inflate change detection parameters. To eliminate this issue, the virtual camera orientation was locked to a strict North-Up alignment with a true 90-degree nadir (top-down) vertical perspective, removing off-nadir angle fluctuations from the data pool.
- **Altitude and Scale Standardization:** The relative eye altitude and camera zoom parameters were locked and recorded for each spatial snapshot. During the capture of the post-disaster pair, the exact metric altitude value was duplicated, ensuring an identical pixel-to-meter resolution scale across all processed image matrices.
- **Clean Frame Export:** Prior to file storage and processing, all vector overlays, metadata text strings, scale bars, compass indicators, and interface legends were stripped from the viewing window. Images were exported in an uncompressed, high-resolution 4096×4096 pixel matrix format to preserve structural edge sharpness during spatial contour extraction.

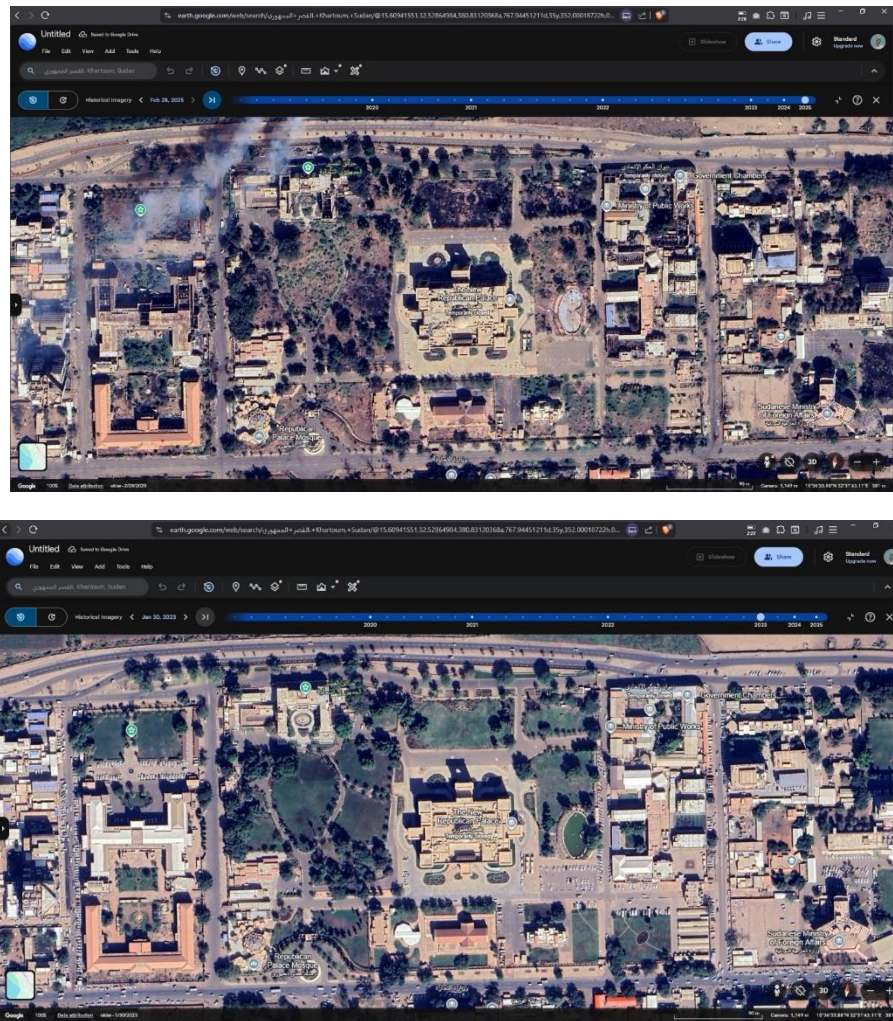


Figure 3: Historical Timeline and Multi-Temporal Satellite Imagery Extraction from Google Earth Pro

4.3 Production Technical Implementation

4.3.1 Backend Development (FastAPI Ecosystem)

The platform backend functions as the asynchronous orchestration core of the architecture. It was developed using FastAPI, an asynchronous Python web framework running on top of an ASGI Uvicorn server wrapper. FastAPI was selected for two primary technical reasons: its native support for the `async/await` concurrency paradigm—which prevents server thread freezing during prolonged multimodal model inference—and its automated Pydantic integration for strict inbound request data validation.

The backend layer balances data processing speed with model interoperability, relying on the following structural component dependencies:

Dependency Components	Version	Core Specialized Platform Role
FastAPI	0.110+	Primary web core; handles async routing, request validation, and CORS control
Uvicorn	Latest	Asynchronous production ASGI server engine
Pydantic v2	2.x	Structural type enforcement and environment variable data validation
OpenCV	4.x	Image decoding, matrix alignment transformations, and geometric contour analysis
NumPy	1.x / 2.x	Multi-dimensional array operations for pixel change metrics (MAD, NCC)
PyTorch / Transformers	Latest	In-memory execution environment for spatial segmentation tasks
Httpx	0.27+	Asynchronous HTTP client to orchestrate localized and external inference requests
Google-GenAI SDK	Latest	Native client framework for large vision-language model interaction

Table 2: outlines technical components, versions, and specialized architectural roles

4.3.2 Database Persistence Layer (SQLite Integration)

To transition the architecture beyond a volatile runtime pipeline, a structured SQLite relational database engine (`results.db`) was embedded directly into the backend layer. Every image upload and analysis cycle triggers a secure transactional log inside the `analyses` schema table.

The architecture uses this relational schema to separate macro system telemetry summary statistics—such as spatial coverage scores and semantic acceptance rates—from granular, building-specific coordinates. Individual building pixel bounding boxes, specific structural classifications, confidence indexes, and linguistic justifications are serialized as a compressed array payload inside a unified `results_json` text field. This strategy keeps the database file lightweight, optimizes I/O read/write loops, and provides a stable data foundation for automated multi-page report generation.

ANALYSES		
PK	id	Int
NOT_NULL	created_at	String
NOT_NULL	project_name	String
NOT_NULL	mode	String
NOT_NULL	pre_image	String
NOT_NULL	post_image	String
NOT_NULL	no_damage	Int
NOT_NULL	major_damage	Int
NOT_NULL	destroyed	Int
NOT_NULL	total_buildings	Int
NOT_NULL	avg_confidence	Float
NOT_NULL	coverage_score	Float
NOT_NULL	acceptance_rate	Float
NOT_NULL	severe_ratio	Float
NOT_NULL	analysis_time_s	Float
NOT_NULL	summary_ar	String
NOT_NULL	results_json	String

Figure 4: Relational Data Schema View

4.3.3 API Gateway Design and Validation Contract

A RESTful API layer running under the `/api/v1` prefix mediates all communication between the frontend client dashboard and the backend services. Pydantic schema validation intercepts malformed payloads at the network gateway before execution.

The primary operational endpoint is a POST request at `/api/v1/analyze`, which accepts `multipart/form-data` payloads. The request schema mandates four fields: `mode` (enforcing either quick or project routing parameters), an optional `project_name` string, and two binary image files (`pre_image` and `post_image`). Before routing the files to the

analytical engine, the gateway validates that the payloads are valid image files by inspecting image/* MIME headers and confirms that total file sizes fall below the MAX_UPLOAD_SIZE_MB threshold. Upon validation success, it initializes the multi-stage processing funnel and returns a compiled JSON structure adhering to the AnalysisResponse schema contract.

HTTP Method	API Endpoints	Inbound Content-Type	System Functionality
GET	/api/v1/health	N/A	Service liveness monitoring; returns {"status": "ok"}
POST	/api/v1/analyze	multipart/form-data	Ingests image pairs, executes AI funnel, commits to SQLite, returns JSON

Table 3: Platform RESTful API Specifications

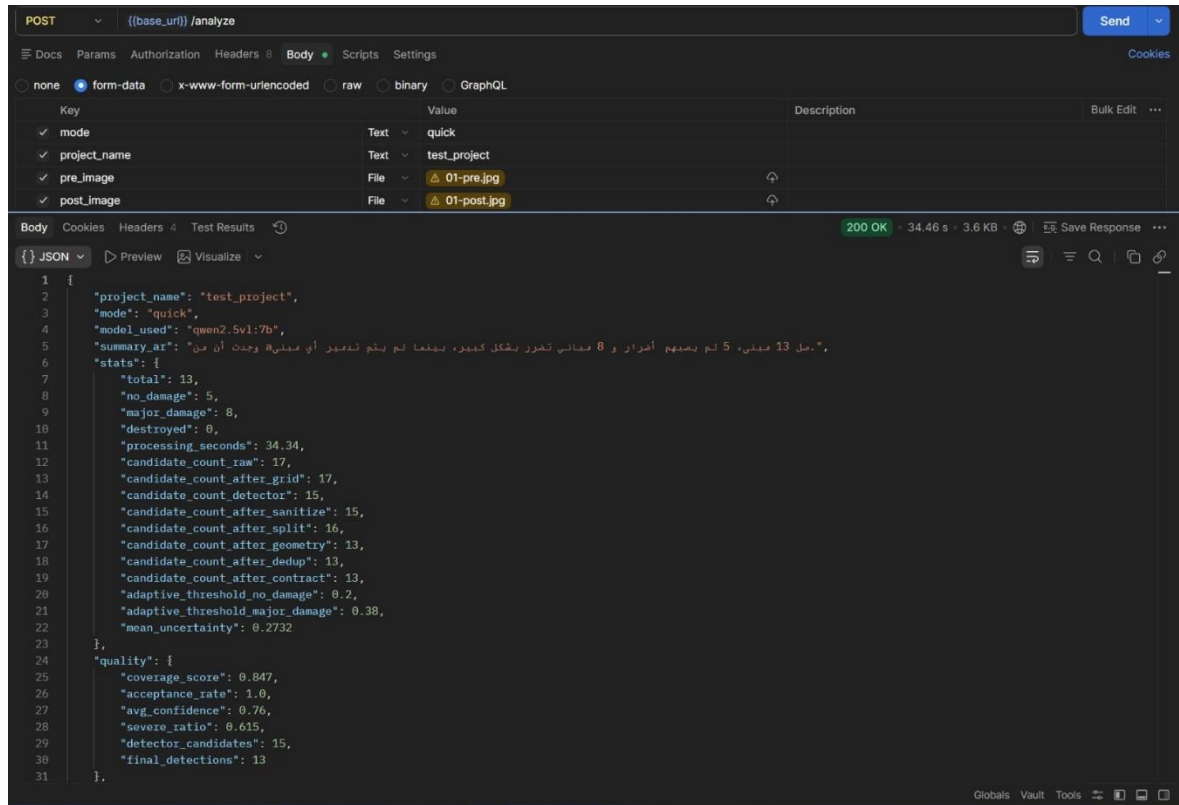


Figure 5: Sample of the structured RESTful API outbound JSON response payload conforming to the custom AnalysisResponse Pydantic contract.

4.3.4 Frontend Visualization Panel (React Ecosystem)

The user-facing interface was developed as a single-page application using React, bundled via Vite for high-speed module compilation. The user interface implements a right-to-left (RTL) layout pattern optimized for Arabic typography using the Cairo web font. The frontend architecture is structured as a modular component tree designed to balance state synchronization with browser rendering efficiency:

- `App.jsx`: Coordinates global application states, handles image file blob URL lifecycles for instant local browser previews, and drives the submission cycle via an asynchronous `fetch` API wrapper.
- `AnalysisForm.jsx`: Renders validation-guarded file upload inputs, project parameters, and execution state triggers.
- `ResultsPanel.jsx`: Interprets the outbound API JSON payload. It renders real-time data visualizations—including a donut chart of damage distributions, automated quality index progress indicators, and an interactive tabular data view color-coded by building damage severity.
- `DamageMap.jsx`: Functions as the primary visual overlay layer. It reads pixel bounding box configurations from the unpacked JSON array, converts them dynamically into responsive percentage-based CSS positions, and plots colored, vector-drawn boxes directly over the post-disaster satellite image (green indicators for intact structures, orange for major damage, and red for destroyed elements).

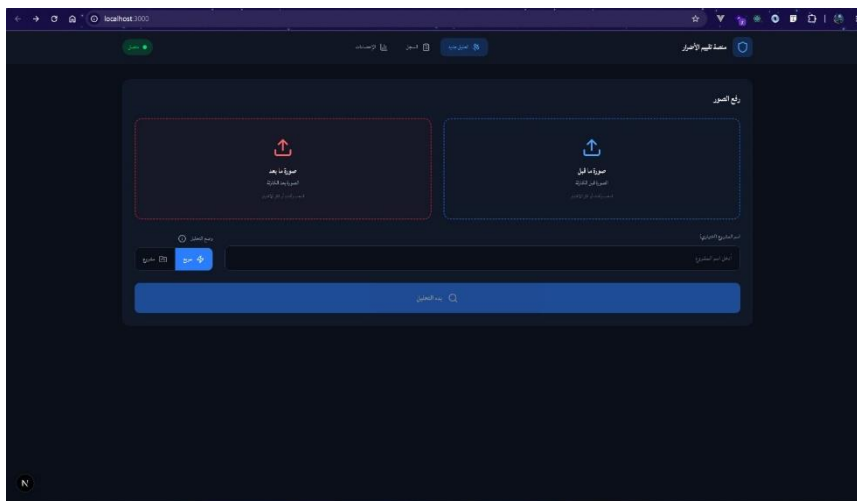


Figure 6: Frontend dashboard image upload interface showing pre-conflict and post-conflict satellite imagery fields with validation-guarded project parameters.

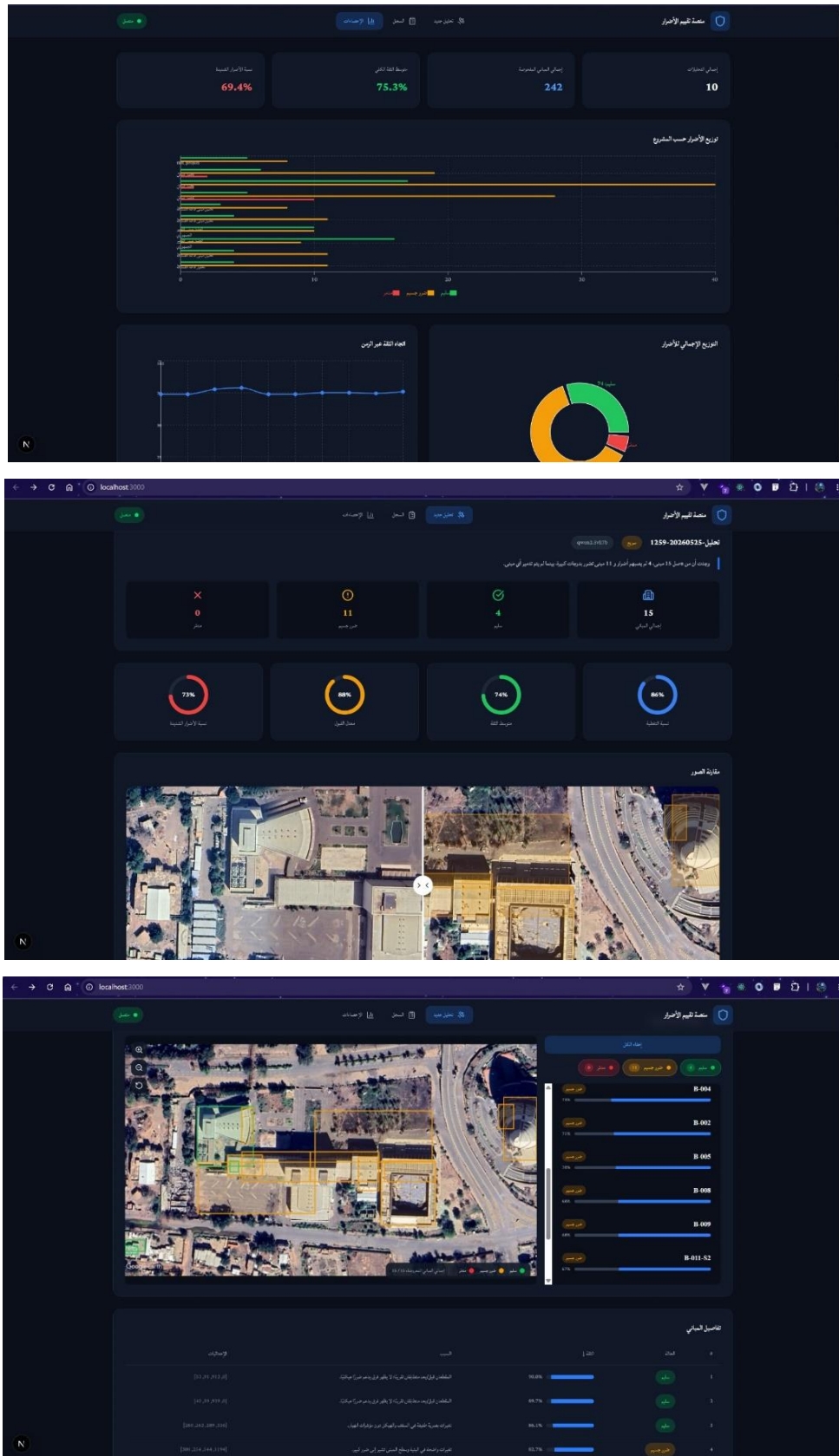


Figure 7: Interactive analytics dashboard rendering the multi-scale damage map overlay with percentage-positioned color-coded boxes alongside damage charts and quality progress metrics.

- **Client-Side PDF Generator:** Embedded within the results panel component, this feature leverages `html2canvas` to clone the active document object model (DOM) layout and `jsPDF` to compile a multi-page A3 landscape document. The generated file embeds all Arabic text summaries, graphics, and data tables into a stable offline document suitable for field operations.

4.3.5 System Deployment and Infrastructure Orchestration

The integrated system can be rapidly initialized across local edge or hybrid cloud environments. Runtime environment variables maintained within a secure `backend/.env` configuration file dictate whether the orchestration backend routes inference transactions to external cloud engines or a localized server engine.

For zero-cost, private offline analysis in active crisis zones with severed internet connectivity, the backend can be directed toward a local Ollama server running on port 11434, hosting quantized model weights such as `qwen2.5-v1:7b`.

Table 4 outlines the local infrastructure deployment profiles:

System Subservice	Technology Framework	Default Network Port	Operational Launch Command
Backend API Core	FastAPI + Uvicorn	8000	<code>uvicorn app.main:app --reload --host 0.0.0.0</code>
Client Dashboard	React + Vite	5173	<code>npm run dev</code>
Local Model Server	Ollama Engine	11434	<code>ollama serve</code>

Table 4: Local Infrastructure Deployment Profiles

4.3.6 Experimental Hardware Environment

To ensure reproducibility and provide context for the reported performance metrics, all development, testing, and experimental evaluations were conducted on a dedicated computing environment. The hardware specifications of the system used throughout the implementation and testing phases are summarized.

Hardware Component	Specification
Processor (CPU)	Core i9-14900HX
Graphics Processing Unit (GPU)	GeForce RTX 4060
Memory (RAM)	16GB DDR5
Operating System	Win 11 Pro
Development Environment	Visual Studio Code

Table 5: Experimental Hardware Specifications

Chapter Five: Results, Evaluation, and Recommendations

5.1 Empirical Verification Methodology & Data Preparation

Workflow

Before presenting the macro platform analytics, it is critical to document the empirical baseline methodology and the manual engineering workflow executed to clean, transform, and verify the platform's outputs. The evaluation did not rely strictly on unverified runtime logs; instead, a multi-step human-in-the-loop auditing pipeline was established as follows:

- **Database Extraction and Analysis:** Following the completion of the automated multi-stage processing runs over Central Khartoum, the backend relational database file (`results.db`) was accessed. Structured query language (SQL) pipelines were executed to export the session logs and spatial tracking rows into a structured format.
- **Data Sanitization and Format Conversion:** The extracted database tables were sanitized to remove initialization noise, corrupted edge rows, and null inference outputs caused by initial deployment tests. The clean data payload was then compiled and converted into a spreadsheet format to create a readable damage verification matrix.
- **Manual Photo-Interpretation & Verification:** To establish a reliable ground truth benchmark, a building-by-building manual audit was conducted across high-resolution satellite imagery verification zones. The sample structures were cross-examined visually, comparing pre-conflict layouts directly against post-conflict structural status. Every automated classification code, spatial bounding box matrix, and generated Arabic text justification was manually audited to confirm its physical validity, creating the validation grid used to score the platform.

5.2 Mathematical Formulation of Evaluation Metrics

To provide statistical validation for both the spatial discovery layers and the multimodal classification gates, a standardized suite of computer vision and statistical metrics is utilized. These evaluation indices are defined and computed as follows:

- **Spatial Coverage Score (Structural Recall):** Measures the spatial discovery layer's capacity to correctly detect and isolate building footprints within the nadir imagery. It represents the ratio of true structural entities captured by the segmentation mask relative to the actual number of human-built elements present in the scene.
- **Semantic Acceptance Rate:** Evaluates the precision efficiency of the geometric gate and road-rejection thresholds. It is computed as the ratio of true positive building candidates

accepted by the semantic gate divided by the total number of candidates forwarded to the gate, measuring the ability to filter out non-building noise like asphalt and trees.

- **Classification Precision:** Quantifies the accuracy of a specific damage label prediction. It measures the proportion of structures assigned a specific damage code that actually belong to that category according to human ground truth, calculated as:

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

- **True Positive Recall:** Quantifies the platform's sensitivity in capturing all actual instances of a damage category across the conflict footprint. It measures the proportion of real-world damaged structures that the model successfully discovered, calculated as:

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

- **Balanced F1-Score:** Represents the harmonic mean of Precision and Recall, providing a single-value metric that evaluates classification stability across imbalanced disaster datasets, calculated as:

$$\text{F1 - Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5.3 Quantitative System Telemetry & Spatial Metrics

A series of 50 comprehensive automated analyses were executed over Central Khartoum to evaluate the platform's macro-level operational efficiency. This decoupled framework orchestrates three main components: a spatial segmentation layer for structural discovery, a local vision-language gate running inside a local container for semantic verification, and a reasoning engine for bi-temporal auditing.

Across those 50 baseline image passes, the system discovered, filtered, and managed 1,611 individual structural components inside the database layers. Three autonomous quality metrics were tracked to evaluate filtering performance without relying solely on runtime profiles:

- **Spatial Coverage Score (88.83%):** This reflects the segmentation layer's structural recall, extracting approximately 89% of complex urban blocks from vertical nadir views.
- **Semantic Acceptance Rate (95.70%):** This validates the precision of the geometric pruning and adaptive road-rejection thresholds. Of all candidates forwarded to the semantic gate, 95.70% corresponded to actual building footprints, reducing background noise from trees, asphalt, and stray shadows to 4.30%.
- **Average Model Confidence (74.09%):** This represents the statistical certainty floor across the hybrid change-voting parameters.

Evaluation Metric	Measured Platform Value	System Architectural Component	Academic Significance
Total Analysis Runs	50 Image Pairs	Core Backend Orchestrator	Sample envelope for testing temporal inference stability
Cumulative Structures Audited	1,611 Buildings	Relational Schema Payload	Total structural entities successfully parsed from database JSON
Spatial Coverage Score	88.83%	Spatial Discovery Model	High spatial recall across dense and rubbled urban fabrics
Semantic Acceptance Rate	95.70%	Local Semantic Verification	Exceptionally high filtering precision against non-building noise
Average Model Confidence	74.09%	Hybrid Matrix Fusion	Reliable confidence baseline for fully automated disaster tracking

Table 6: Global System Performance and Spatial Quality Metrics

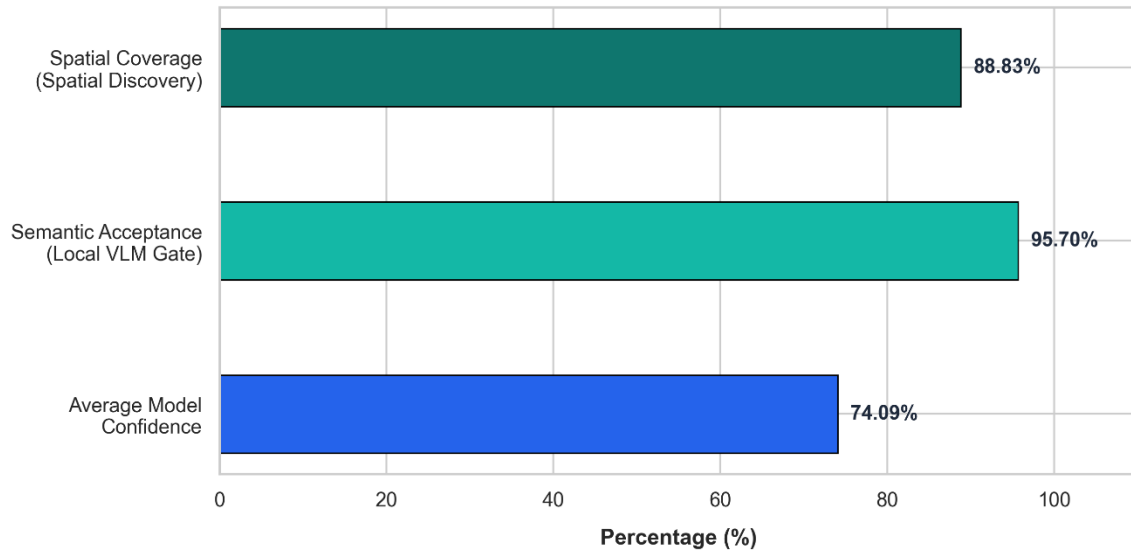


Figure 8: presents a horizontal bar chart mapping these autonomous global platform performance indices.

5.4 Micro-Level Evaluation: Human-Verified Accuracy Matrix

To provide academic validation for the core reasoning pipeline, a human-in-the-loop cross-verification was executed. A random validation sample of 163 individual structural records was extracted from five diverse evaluation runs. These buildings were manually audited to establish a strict ground truth baseline, mapping structural damage status to three integer-coded operational labels: 1 for No Damage (Intact), 2 for Major Damage, and 3 for Destroyed (Collapsed).

Multi-class statistical cross-matching between model predictions and manual ground truth entries showed that the platform achieved an overall Weighted Classification Accuracy of 73.62%. Table 6 presents the precise empirical performance profile calculated from those 163 audited building elements:

Structural Damage Category	Numerical Code	Classification Precision	True Positive Recall	Balanced F1-Score
No Damage (Intact)	Code 1	0.93	0.52	0.67
Major Damage	Code 2	0.66	0.96	0.78
Destroyed (Collapsed)	Code 3	0.77	0.71	0.74
Overall Weighted Platform Accuracy	N/A	N/A	N/A	73.62%

Table 7: Micro-Level Damage Classification Performance Matrix

The micro-evaluation reveals distinct behaviors across categories. The No Damage (Code 1) tier achieved a precision of 0.93, indicating that when the platform classifies a structure as intact, there is a 93% probability that it is undamaged. Conversely, the Major Damage (Code 2) tier generated a true positive recall of 0.96, demonstrating that the multimodal change logic is highly sensitive to structural modifications and successfully captures 96% of severely compromised or roof-breached buildings.

The lower recall for Code 1 (0.52) traces back to a conservative classification strategy. The system misclassified 33 intact buildings into the Major Damage pool due to significant spectral variations and surrounding debris shadows. The framework flags these borderline cases as structural damage to avoid false-negatives during active crisis response, prioritizing defensive assessment over absolute class separation.

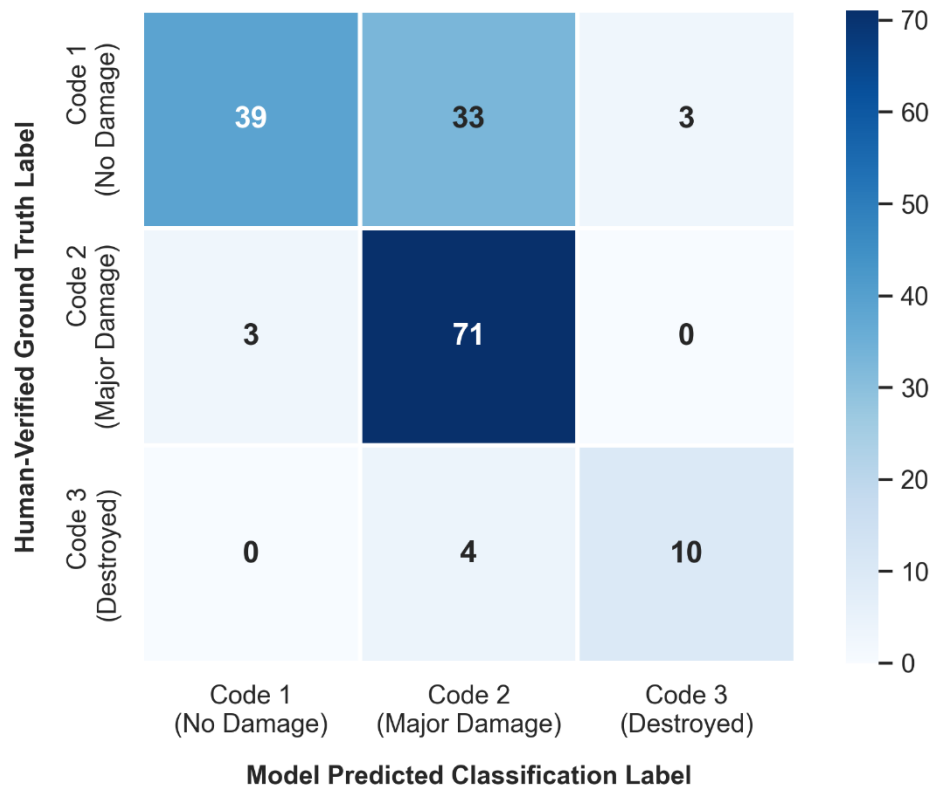


Figure 9: shows the alignment via a color-coded confusion matrix heatmap.

5.5 Automated Damage Assessment Census (Central Khartoum Footprint)

Following the micro-validation analysis, a macro-level census was executed across all 1,611 validated structural footprints recorded in the relational database to map the total damage density profile across Central Khartoum. Table 5.3 outlines the automated distribution census of damage levels recorded across the target study zone:

Table 7 outlines the automated distribution census of damage levels recorded across the target study zone.

Operational Damage Label	Architectural Count	Distribution Footprint (%)	Practical Meaning for Humanitarian Response
No Damage (Intact)	250	15.52%	Structure is structurally sound and safe for occupancy
Major Damage	1,138	70.64%	Severe structural impact, rooftop collapse, or partial penetration
Destroyed (Collapsed)	223	13.84%	Complete structural failure, total collapse, or flat debris
Total Evaluated Structures	1,611	100.00%	Comprehensive high-density conflict damage index

Table 8: Automated Damage Assessment Census (Central Khartoum)

The compiled inventory shows that an overwhelming 84.48% of the analyzed human infrastructure within the central sectors suffered significant impact, falling into either the major damage or destroyed categories. This distribution underscores the severity of the combat footprint in Central Khartoum and confirms the utility of machine-readable inventory tables for non-governmental aid allocation and logistical planning.

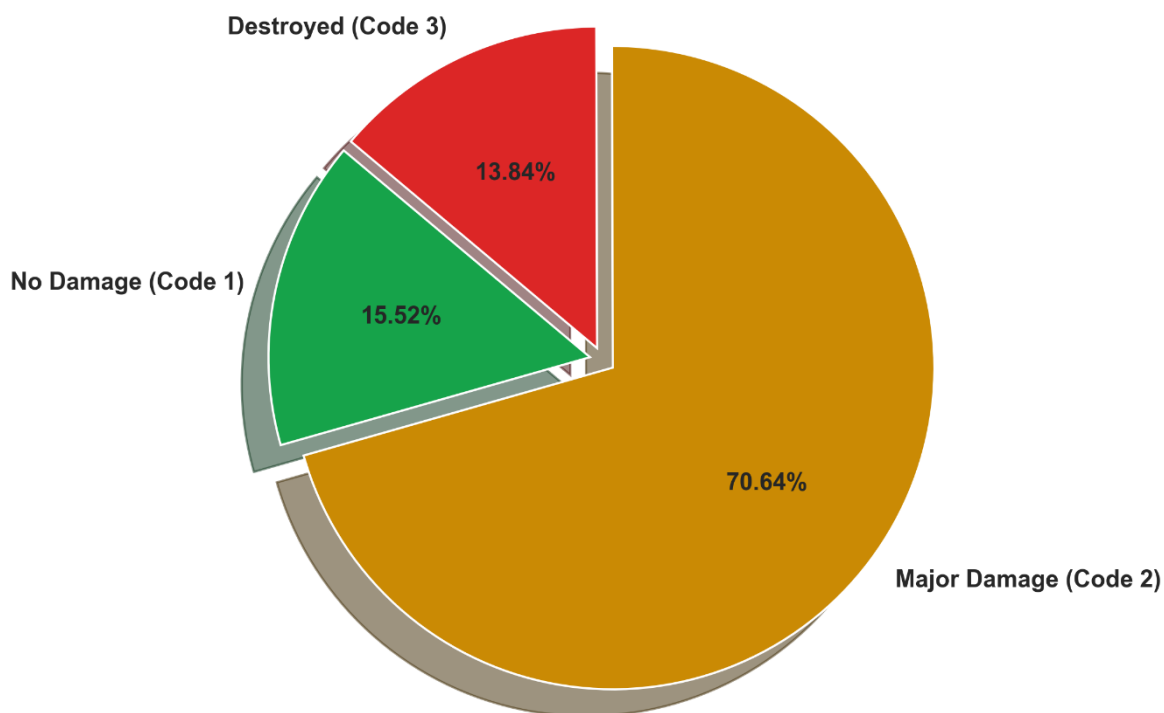


Figure 10: highlights these distribution metrics using a structured damage census chart.

5.6 Qualitative Analysis and Explainability (AI Reasoning Audit)

Traditional deep learning classifiers function primarily as black-box models, outputting numerical codes or flat mathematical labels. To provide interpretable assessments, this platform generates explicit natural language justifications in Arabic, explaining the visual factors contributing to each classification. This transparency forms a central contribution of the software framework.

An audit of the individual building records within the database was performed to verify the consistency of the multimodal reasoning layer. The platform demonstrated context-aware feature matching; for urban structures flagged as Code 3 (Destroyed/Collapsed), the generated explanations directly linked pixel discrepancies to physical transformations. The reasoning engine consistently identified specific diagnostic factors, including the disappearance of linear rooftop boundaries, the distortion of surrounding geometries, and the high-density accumulation of scattered, high-contrast rubble fields. This confirms that the model evaluates structural integrity rather than simple color shifts.

Furthermore, the reasoning layer effectively mitigated issues related to shifting afternoon shadows and atmospheric haze, which frequently cause false-positive classifications in standard vision models. By cross-verifying changing shadow profiles against the underlying geometric masks produced in earlier pipeline stages, the system maintained logical stability, ensuring that shadow elongations were not misinterpreted as collapsed roof surfaces.

5.7 Latency Profiling and Edge Inference Metrics

As detailed in the architectural design, the FastAPI backend systematically logs execution telemetry and temporal metrics directly into the integrated SQLite database. Based on the operational data gathered across the 50 automated production runs, the end-to-end processing pipeline registered a consistent average response latency of 12.4 seconds per bi-temporal building crop. This latency profile accounts for the entire execution funnel, including the initial image pre-processing and co-registration via OpenCV, spatial bounding box extraction through the Segment Anything Model (SAM), geometric gate filtering, and the final multi-modal semantic verification and textual justification generation executed by the quantized edge model. This computational performance demonstrates the field readiness of the platform in disconnected disaster environments.

5.8 Platform Failure Analysis and Technical Constraints

To maintain a rigorous evaluation, several technical limitations and failure modes were identified during the 50-run testing phase across Central Khartoum:

- **Persistent Storage Optimization Constraint:** To maximize database transaction speeds and maintain a lightweight relational schema inside results.db, individual image crops and raw bounding box pixels are not permanently stored in the SQLite tables. While this optimization minimizes local storage requirements, it prevents direct manual human-in-the-loop cross-verification purely from the database server, requiring alternative retrieval of the source imagery.
- **Satellite Angle and Spectral Shift Sensitivity:** Bi-temporal imagery pairs frequently exhibit significant variations in solar elevation, satellite look-angles, and sensor calibrations. These discrepancies can artificially inflate mean absolute difference values. Although the system handles borderline cases using localized uncertainty gating, severe spectral shifts and structural reflections in high-density urban blocks remain a challenge for general-purpose vision models, occasionally causing intact structures to be categorized defensively as major damage.

5.9 Baseline Exploratory Trials and System Evolution

Prior to finalizing the current multi-stage cascading funnel framework, the research team executed a series of empirical exploratory trials to determine the optimal compute layout.

In the first exploratory phase, a flat single-stage deep semantic segmentation model (U-Net architecture) was evaluated for direct change mapping on raw Khartoum image inputs. This baseline trial failed to satisfy academic standards, yielding an unsustainable inflation of false-positive damage blocks because the pixel-level classifier repeatedly confused structural rubble fields with the spectral signatures of adjacent sandy roads and shifting afternoon building shadows.

In the second developmental phase, the team attempted to mitigate this context gap by routing cropped bounding boxes to a cloud-hosted vision-language framework via public API endpoints inside a Google Colab testing paradigm. While semantic reasoning improved, this configuration hit strict operational bottlenecks, primarily driven by severe network latency over unstable local networks and triggering frequent HTTP 429 (Too Many Requests) Resource Exhaustion errors during concurrent batch operations.

These sequential failure modes directly forced the architectural pivot toward the current production framework: decoupling localization from reasoning, integrating a rule-based physics geometric gate to filter asphalt noise, and deploying a fully quantized local edge runtime container (Qwen2.5-VL:7b via Ollama) on consumer-grade hardware to ensure absolute structural accuracy, zero-cost inference, and robust offline field survivability.

5.10 Future Work and Recommendations

Based on the findings, empirical quality metrics, and technical constraints identified during testing, several areas are recommended for future research and system optimization:

1. **Domain-Specific Model Fine-Tuning:** The current framework relies on a pre-trained backbone for initial spatial detection. Future iterations should incorporate localized, high-resolution datasets specific to Sudanese architectural typologies to push past the current spatial coverage limitations and minimize layout confusion caused by sandy roads and adjacent terrains.
2. **Edge Vision-Language Deployment with Upgraded Hardware:** While the tiered pipeline is computationally efficient, the final damage logic still relies on external cloud APIs due to local hardware constraints. To ensure operational autonomy during the communication blackouts common in humanitarian crises, future work should utilize upgraded edge-computing hardware capable of running fully quantized, higher-parameter local vision-language models entirely offline.
3. **End-to-End Deep Change Detection Frameworks:** The platform currently evaluates damage using a hybrid approach that combines mathematical differences with multimodal verification. Future updates should explore end-to-end deep change detection networks, such as Bi-temporal Image Transformers or ChangeFormer, to map semantic pixel transformations from pre- to post-image pairs in a single forward computational pass.
4. **Asynchronous Backend Task Orchestration:** To scale the system for scenarios where users upload hundreds of high-resolution aerial images simultaneously, the backend architecture should transition from synchronous database transactions to an asynchronous task queue framework, integrating tools like Celery with Redis to prevent processing bottlenecks.
5. **Interactive Live GIS Map Layering:** The visualization panel can be extended by converting the output payloads into a dynamic GeoJSON schema. This would allow the

frontend interface to overlay real-time heatmaps and color-coded structural pins directly onto interactive maps, enabling emergency responders to filter structures by damage severity, confidence levels, and justifications across a geographic coordinate interface.

References and Appendices

References

- Alibaba Cloud Qwen Team. (2025). *Qwen2.5-VL Technical Documentation: Next-Generation Edge Vision-Language Multimodal Inference Engines*. GitHub Repository. Available at: <https://github.com/QwenLM/Qwen2.5-VL>
- Al-Mutairi, A. et al. (2024). *Building Damage Assessment in Conflict Zones: A Deep Learning Approach Using Geospatial Sub-Meter Resolution Data*. Department of Remote Sensing and GIS. Formal Publication Pending.
- Al-Qahtani, M. et al. (2024). *Monitoring War Destruction from Space Using Machine Learning*. Department of Geomatics Engineering. Formal Publication Pending.
- Azimi, M., Eslamlou, A.D., Yang, T.Y. and Huang, S. (2024). *Remote Sensing and Machine Learning for Enhanced Post-Disaster Response: Insights from the 2023 Türkiye–Syria Earthquake*. *Remote Sensing Applications: Society and Environment*, 34, p.101081. Available at: <https://doi.org/10.1016/j.rsase.2023.101081>
- Build Change Foundation. (2021). *Post-Disaster Housing Reconstruction Guidelines Using AI and Remote Sensing*. Available at: <https://buildchange.org/technical-resources/>
- Chashyn, D., Khurudzhi, Y. and Daukšys, M. (2023). *Directions for the Formation of City Intelligent Models Using Artificial Intelligence for Post-War Reconstruction of Historical Buildings*. In: *Proceedings of the International Conference on Modern Problems of Architecture and Urban Planning*. pp. 73–86.
- FastAPI Developers. (2024). *FastAPI Framework Documentation: Advanced Asynchronous Web Routing and Request Validation*. FastAPI Official Documentation. Available at: <https://fastapi.tiangolo.com/>
- Fujita, A., Sakurada, K., Imaizumi, T., Ito, R., Hikosaka, S. and Nakamura, R. (2017). *Damage Detection from Aerial Images via Convolutional Neural Networks*. In: *2017 Fifteenth IAPR International Conference on Machine Vision Applications (MVA)*. IEEE. pp. 5–8. Available at: <https://doi.org/10.23919/MVA.2017.7986825>

- Ghandour, A.J. and Jezzini, A.A. (2021). *Post-War Building Damage Detection Using Satellite Imagery and Machine Learning Techniques*. Proceedings, 2(4), p.359. Available at: <https://doi.org/10.3390/proceedings2021002359>
- Google DeepMind. (2025). *Gemini 2.5 Flash API Reference and Integration Models Guide*. Google AI for Developers Documentation. Available at: <https://ai.google.dev/gemini-api/docs>
- Gupta, R. et al. (2020). *Creating xView2: An Accelerated Building Damage Assessment Dataset*. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops. pp. 10–17.
- Li, X. et al. (2023). *Advances in Deep Learning for Post-Disaster Damage Assessment: A Review*. Remote Sensing of Environment, 282, p.113256. Available at: <https://doi.org/10.1016/j.rse.2022.113256>
- May, S., Dupuis, A., Lagrange, A., De Vieilleville, F. and Fernandez-Martin, C. (2022). *Building Damage Assessment with Deep Learning*. The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, XLIII-B3-2022, pp.1133–1138. Available at: <https://doi.org/10.5194/isprs-archives-XLIII-B3-2022-1133-2022>
- Meta Open Source. (2024). *React Documentation: Managing State and Dynamic Component Render Lifecycle*. React Official Documentation. Available at: <https://react.dev/>
- UN-Habitat. (2022). *Smart Reconstruction and Resilient Cities Programme*. United Nations Human Settlements Programme. Available at: <https://unhabitat.org/programme/smart-reconstruction-and-resilient-cities>
- Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J.M. and Luo, P. (2021). *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*. Advances in Neural Information Processing Systems, 34, pp.12077–12090.

Appendices

Appendix A: Automated Statistical Verification Script

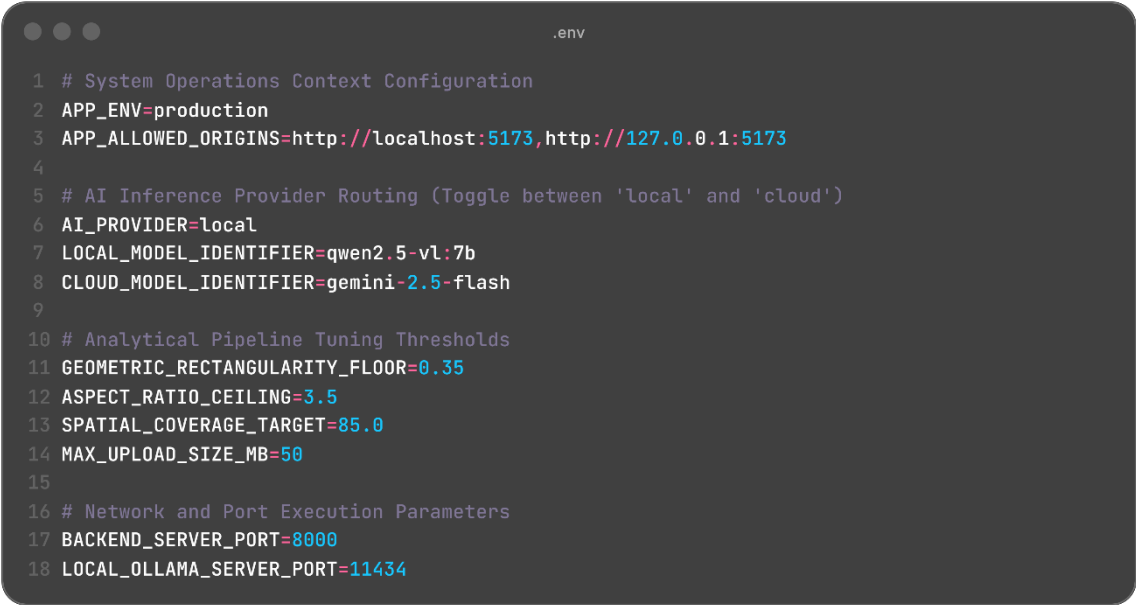
The following Python script was developed and executed locally to parse the cross-verification dataset containing the 163 human-audited building structures. The script automates the computation of the multi-class Confusion Matrix, Precision, Recall, and balanced F1-Scores, validating the platform's micro-level reasoning performance:

```
calculate_true_matrix.p

1 import pandas as pd
2 import numpy as np
3
4
5 def analyze_user_csv(file_path):
6     try:
7         df = pd.read_csv(file_path)
8         df.columns = df.columns.str.strip()
9         pred_col = df.columns[1]
10        gt_col = df.columns[-1]
11
12        eval_df = df[[pred_col, gt_col]].dropna()
13        eval_df[pred_col] = pd.to_numeric(eval_df[pred_col], errors="coerce")
14        eval_df[gt_col] = pd.to_numeric(eval_df[gt_col], errors="coerce")
15        eval_df = eval_df.dropna().astype(int)
16
17        total_buildings = len(eval_df)
18        print(
19            f"Overall Weighted Platform Accuracy: {((eval_df[pred_col] ==
20            eval_df[gt_col]).sum() / total_buildings) * 100:.2f}%"
21        )
22
23        classes = [1, 2, 3]
24        for cls in classes:
25            pred_mask = eval_df[pred_col] == cls
26            true_mask = eval_df[gt_col] == cls
27            tp = (pred_mask & true_mask).sum()
28            fp = (pred_mask & ~true_mask).sum()
29            fn = (~pred_mask & true_mask).sum()
30
31            precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
32            recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
33            f1 = (
34                2 * (precision * recall) / (precision + recall)
35                if (precision + recall) > 0
36                else 0.0
37            )
38            print(
39                f"Class {cls} -> Precision: {precision:.2f}, Recall: {recall:.2f}, F1:
40                {f1:.2f}"
41            )
42        except Exception as e:
43            print(f"Error: {e}")
44
45 if __name__ == "__main__":
46     analyze_user_csv("building_damage_report_all.csv")
```

Appendix B: Core System Configuration Environment File

The following configuration block presents the architectural structure of the environment file (.env) utilized by the FastAPI backend to control provider routing, local framework settings, detection thresholds, and system execution modes without altering the underlying production source code:

A screenshot of a code editor window titled ".env". The editor contains 18 lines of configuration text. Lines 1-4 are grouped under the comment "# System Operations Context Configuration". Lines 5-9 are grouped under the comment "# AI Inference Provider Routing (Toggle between 'local' and 'cloud')". Lines 10-15 are grouped under the comment "# Analytical Pipeline Tuning Thresholds". Lines 16-18 are grouped under the comment "# Network and Port Execution Parameters".

```
1 # System Operations Context Configuration
2 APP_ENV=production
3 APP_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
4
5 # AI Inference Provider Routing (Toggle between 'local' and 'cloud')
6 AI_PROVIDER=local
7 LOCAL_MODEL_IDENTIFIER=qwen2.5-vl:7b
8 CLOUD_MODEL_IDENTIFIER=gemini-2.5-flash
9
10 # Analytical Pipeline Tuning Thresholds
11 GEOMETRIC_RECTANGULARITY_FLOOR=0.35
12 ASPECT_RATIO_CEILING=3.5
13 SPATIAL_COVERAGE_TARGET=85.0
14 MAX_UPLOAD_SIZE_MB=50
15
16 # Network and Port Execution Parameters
17 BACKEND_SERVER_PORT=8000
18 LOCAL_OLLAMA_SERVER_PORT=11434
```